

Enterprise Agentic AI Development Lab

Open-Source-First, Air-Gap-Ready Evidence-to-Application Workflow

Evergreen Learning Report

Version 0.1 | 7 June 2026

Prepared for internal knowledge sharing

Primary learning focus	Convert unstructured documents into traceable, approved specifications before coding begins.
Development approach	4D product lifecycle, spec-driven development, user stories, test-driven validation and human approval gates.
Enterprise constraint	Prefer open-source and air-gap-compatible components; use OpenAI only as a personal-lab development accelerator.

Document Control

Field	Value
Document status	Evergreen working document
Current version	0.1
Date	7 June 2026
Owner	Keni Lim
Update rule	Revise this document after each meaningful learning point, architecture decision or validated experiment.
Intended audience	Colleagues assessing enterprise agentic product-development approaches in air-gapped or restricted environments.

Version History

Version	Date	Summary of change
0.1	7 June 2026	Initial export. Captures the lab purpose, target workflow, completed setup, local-model experiment results, GitHub-backup status and next implementation steps.

Contents

- 1. Executive Summary
- 2. What We Are Trying to Build
- 3. Target 4D Product-Development Workflow
- 4. Comparison with the Current Proposed Squad Design
- 5. Open-Source-First Architecture
- 6. Hands-On Lab Environment Completed So Far
- 7. Experiment Results and Learning Points
- 8. Unstructured-Document Ingestion Strategy
- 9. Specification, User-Story and Test Traceability Model
- 10. Human-Triggered Automation and Future Agent Roles
- 11. GitHub Backup and Repository Strategy
- 12. Logging and Token-Usage Measurement
- 13. Roadmap
- Appendix A. Current Folder Structure
- Appendix B. Tooling Register
- Appendix C. Source References

1. Executive Summary

This lab is a hands-on investigation into the best way to convert messy enterprise evidence - such as emails, PDFs, Word files, PowerPoint decks, spreadsheets, scanned pages and screenshots - into a workable, tested application through an agent-assisted product-development lifecycle.

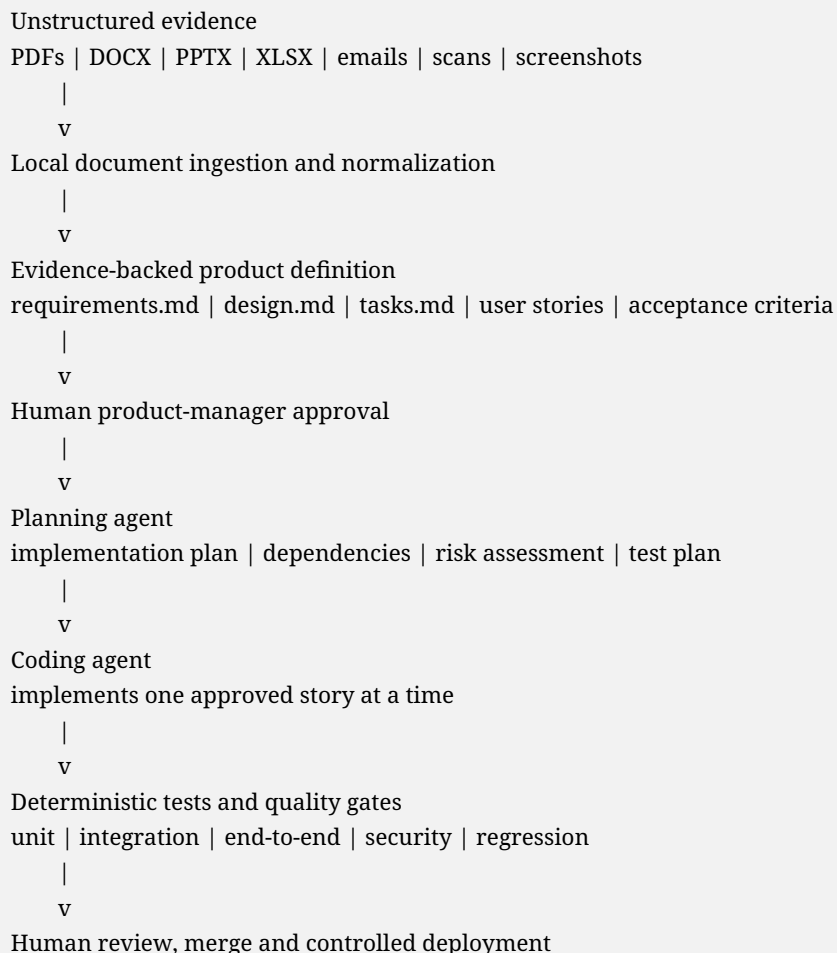
The central design principle is that raw evidence must not flow directly into autonomous coding. The system should first convert unstructured information into a versioned product contract: requirements, design decisions, implementation tasks, user stories, acceptance criteria and test cases. Humans approve this contract before a coding agent begins implementation.

The personal lab deliberately uses an open-source-first architecture so that most of the workflow can later be reproduced inside an air-gapped enterprise environment. OpenAI models are treated as a temporary personal-lab exception for development acceleration, not as a dependency of the enterprise target state.

The experiment has already validated the Windows-to-WSL development environment, Docker integration, local Ollama inference, GPU use, structured logging and token snapshots. It has also surfaced a real integration defect: local Ollama models respond correctly through direct API calls, but the tested OpenCode-to-Ollama adapter does not reliably surface final responses. This is a useful enterprise lesson: model-server validation and agent-framework validation are separate gates.

2. What We Are Trying to Build

The objective is an evidence-to-application workflow that mirrors a real product-development lifecycle rather than a one-shot 'vibe coding' workflow.



The immediate learning focus is the first half of the flow: how to read unstructured documents reliably, preserve source evidence, draft specifications locally and convert the resulting work into small user stories that can be built and tested.

3. Target 4D Product-Development Workflow

The lab uses the traditional 4D structure - Discover, Define, Develop, and Test & Deploy - but adds traceability, explicit approval gates and deterministic quality controls.

4D stage	Primary activities	Required outputs	Human control point
Discover	Ingest evidence, identify business processes, map pain points, detect conflicts and missing information.	Evidence register, source references, process map, stakeholder map, open questions.	Product manager validates the problem statement and evidence quality.
Define	Compile requirements, design decisions, user stories, acceptance criteria, non-functional requirements and risks.	Versioned product contract: requirements.md, design.md, tasks.md, user stories and test traceability.	Product manager and relevant reviewers approve the contract before coding.
Develop	Plan the work, implement bounded stories, review code and resolve issues.	Feature branches, commits, merge requests, code diffs, updated tests and documentation.	Engineer reviews the plan and code changes before merge.
Test & Deploy	Execute unit, integration, end-to-end, regression, security and deployment checks.	Test results, signed release artefact, deployment record, rollback plan and release notes.	Risk-based review before staging and production deployment.

The key control is the traceability spine:

Source evidence

- > Requirement
- > User story
 - > Acceptance criteria
- > Implementation task
 - > Code commit
- > Test case
- > Release

4. Comparison with the Current Proposed Squad Design

The current proposed squad design is directionally strong. It already separates product-manager-led discovery from engineer-led development, uses human review gates, includes planning and development agents, and recognises testing, regression, security and deployment activities.

The main improvement is to make the workflow less agent-centric and more contract-centric. The strongest enterprise pattern is not a large swarm of loosely governed agents. It is a governed evidence plane, a versioned product contract, bounded engineering agents, deterministic CI/CD and end-to-end traceability.

Area	Current proposed approach	Recommended refinement
Discovery	Unstructured documents and user research flow into product agents and prototype generation.	Add a governed evidence layer first: immutable source files, metadata, classification, access controls, hashes, parsing confidence and source citations.
Definition	PRD, prototype and user stories are created before development.	Formalise a versioned product contract in Git: requirements, design, tasks, non-functional requirements, ADRs, acceptance criteria and test traceability.
Agent design	Many named agents and subagents perform specialist activities.	Use a smaller set of reusable roles: product, planner, builder, reviewer, test, security and resolver. Treat specialist capabilities as skills or tools.
Testing	Test, code-fix, regression, security and performance agents are present.	Make CI/CD controls deterministic. Agents may propose fixes, but cannot waive failures or self-approve.
Deployment	Human reviews precede test and production deployment.	Add signed artefacts, SBOMs, release provenance, rollback, monitoring and policy enforcement.
Air-gap operations	Open-source engineering tools and dedicated GPU infrastructure are assumed.	Add a controlled import lane for models, packages, vulnerability feeds, containers and approved tool versions.

5. Open-Source-First Architecture

The standing rule for the lab is: other than OpenAI models used temporarily for personal development, prefer open-source components that can be imported, installed and operated inside an air-gapped enterprise environment.

Open-weight models must still undergo licence review. 'Open-weight' is not automatically equivalent to 'open source'.

PERSONAL LAB

Local evidence route:

documents -> Docling + format-specific extractors -> local model -> draft specs

Development route:

approved synthetic or sanitised specs -> OpenCode -> OpenAI coding model -> code

ENTERPRISE TARGET

Local evidence route:

documents -> same local ingestion stack -> approved local model -> draft specs

Development route:

approved specs -> OpenCode or approved coding interface
-> internal model gateway

-> validated local coding model on enterprise GPUs

5.1 Current and Candidate Components

Component	Role	Classification	Enterprise position	Notes
OpenCode	Coding-agent interface	Open source	Candidate	Supports multiple providers, local models, permissions and project instructions.
Docling	Document parsing and normalization	Open source	Candidate	Supports multiple document formats and advanced PDF understanding.
OpenSpec	Spec-driven workflow	Open source	Candidate	Provides proposal -> specs -> design -> tasks -> implement workflow.
Python	Local orchestration	Open source	Candidate	Used for deterministic ingestion scripts, logging and fallbacks.
pandas / openpyxl	Spreadsheet extraction	Open source	Candidate	Use when spreadsheet semantics, formulas or workbook structure matter.
python-docx / python-pptx	Office-format fallbacks	Open source	Candidate	Use when embedded objects or Office-specific structure need explicit extraction.
Ollama	Workstation model runner	Local-lab convenience	Evaluate	Useful for local experiments; enterprise inference server may differ.
OpenAI models	Development acceleration	Proprietary exception	Replace	Personal lab only. Do not make the enterprise workflow depend on it.
ChatGPT OAuth plugin	Personal OpenCode authentication	Personal-lab exception	Replace	Not an enterprise authentication design.
Docker Desktop / WSL2	Windows workstation development	Personal-lab convenience	Replace or review	Enterprise target should use approved Linux infrastructure and container runtime.

6. Hands-On Lab Environment Completed So Far

Layer	Validated state
Workstation	Windows PC with AMD Ryzen 9 7900, approximately 32 GB RAM and NVIDIA RTX 4070 Ti with approximately 12 GB VRAM.

Layer	Validated state
Linux development layer	WSL2 with Ubuntu 24.04 installed and used as the development workspace.
Workspace	/home/kenilim/projects/enterprise-ai-lab
Editor	VS Code connected to Ubuntu through WSL.
Containers	Docker Desktop integrated with Ubuntu; hello-world container test succeeded.
Developer tooling	Git, Python, pip, virtual environments, Node.js, npm and OpenCode installed.
Local model serving	Windows-hosted Ollama exposed to WSL; local models visible from Ubuntu.
Cloud coding route	OpenCode authenticated to OpenAI through a personal ChatGPT OAuth route for personal-lab development only.
Governance scaffolding	Open-source-first rule, tooling register, architecture-decision records, timestamped logs and usage snapshots created.

7. Experiment Results and Learning Points

7.1 Local Inference Path

The direct local inference route works:

```

Ubuntu WSL
-> Windows Ollama API
-> local open-weight model
-> RTX 4070 Ti

```

The qwen3:14b model was loaded through Ollama. The experiment recorded approximately 8.4 GB loaded into VRAM and approximately 14.9 GB total model size, indicating that the model exceeds available VRAM and is likely partially offloaded. This is acceptable for learning but may reduce speed.

7.2 OpenCode-to-Ollama Adapter Defect

Direct Ollama native and OpenAI-compatible API requests succeeded, including streaming responses. However, OpenCode 1.16.2 did not reliably surface the final assistant answer when using the tested Ollama provider route.

An isolated raw-event test showed reasoning events but no final assistant-text event. The official Ollama-generated OpenCode payload used the same provider adapter and endpoint style as the manual configuration. The decision was to retain Ollama for direct local scripts and use OpenAI models temporarily for OpenCode-driven development.

Enterprise implication: validate each model-server-agent combination independently. A model working through curl is not proof that it works correctly through an agent framework.

7.3 Usage Baseline

Metric	Recorded baseline
Sessions	5
Messages	10
Input tokens	20.3K
Output tokens	360
Primary recorded local models	ollama/qwen3:14b and ollama/mistral:latest
Reported monetary cost	\$0.00 for local inference
Caveat	This baseline includes earlier troubleshooting sessions and is not yet a clean per-project baseline.

7.4 Learning Points to Date

ID	Learning point	Practical implication
LP-001	Linux-style development on Windows	WSL2 and Ubuntu provide a practical Linux workspace on a Windows PC.
LP-002	Containerised development	Docker Desktop works from Ubuntu after enabling integration and Docker-group permissions.
LP-003	Local model serving	Windows Ollama can serve local models to WSL and use the RTX 4070 Ti.
LP-004	Adapter compatibility	Direct model APIs and agent-framework adapters require separate validation.
LP-005	Dual-provider configuration	OpenCode can retain a local provider while using an OpenAI route for personal development.
LP-006	Workspace organisation	Logs, shared assets and pilot projects must be separated early.
LP-007	Human-triggered automation	A deterministic script should process new documents before adding an autonomous agent wrapper.
LP-008	Backup discipline	Local files are not backed up until Git is initialised, a remote repository is created and changes are pushed.

8. Unstructured-Document Ingestion Strategy

Docling is the preferred first-pass ingestion engine because it supports PDF, DOCX, XLSX, PPTX and common image formats, and it provides advanced PDF understanding such as page layout, reading order and table structure. It can export structured outputs such as Markdown and lossless JSON.

Docling should not be treated as the only extractor. The production pattern is a layered ingestion pipeline with format-specific fallbacks and validation.

Input type	First-pass treatment	Fallback or validation
Native PDF	Docling text, layout, reading order and tables	Validate extracted tables and page references against rendered pages.
Scanned PDF or image	Docling OCR and image handling	Use OCR confidence thresholds; send visual extracts to a local vision model where needed.
DOCX	Docling conversion	Use python-docx to inspect embedded images, tables, paragraphs and relationships when fidelity matters.
PPTX	Docling conversion	Use python-pptx to inspect slides, shapes, speaker notes and embedded media where needed.
XLSX	Docling conversion for broad ingestion	Use openpyxl and pandas for workbook structure, sheet-level semantics, formulas, merged cells and data analysis.
Charts, OLE objects and embedded artefacts	Preserve original file and export references where possible	Use file-type-specific inspection; do not assume a generic parser captured all semantics.

8.1 Evidence Package

Each input file should become an evidence package rather than a loose text chunk:

- Original immutable file
- File hash and metadata
- Classification and access-control metadata
- Structured extraction in JSON
- Readable Markdown rendering
- Exported images, tables and figures where relevant
- Source references at document, page, slide, sheet and section level
- Extraction warnings, OCR confidence and fallback decisions

9. Specification, User-Story and Test Traceability Model

The target workflow is spec-driven and test-driven. The coding agent must build from approved user stories, and tests must verify the acceptance criteria attached to those stories.

EVID-0042	Source document, page or sheet reference
REQ-0137	Requirement
STORY-0088	User story
AC-0088-1	Acceptance criterion
TASK-0214	Implementation task
TEST-0045	Unit / integration / end-to-end test

COMMIT Code change
|
RELEASE Deployment artefact

When a test fails, the system should not blindly patch code until CI turns green. It should check whether the failure arises from implementation, test design, requirement ambiguity or a stale plan.

Failure type	Correct response
Implementation defect	Coding agent proposes a bounded code fix and reruns the relevant tests.
Incorrect or incomplete test	Reviewer checks the acceptance criterion and updates the test only with approval.
Ambiguous user story	Return to the requirement and clarify the story before continuing.
Design mismatch	Revisit design.md and update the implementation plan before making broad code changes.
Requirement change	Update the requirement, dependent stories, design, tasks and tests together.

9.1 Minimum Versioned Product Contract

- requirements.md - business requirements, scope, constraints, assumptions and exclusions
- design.md - architecture, interfaces, data model, security considerations and design decisions
- tasks.md - small implementation tasks linked to stories
- user-stories.md - story IDs, acceptance criteria and dependencies
- test-plan.md - test cases linked to story and requirement IDs
- traceability.csv or traceability.json - evidence-to-release linkage
- ADRs - architecture decisions and rationale

10. Human-Triggered Automation and Future Agent Roles

New files should eventually trigger a repeatable local document-to-spec pipeline after an explicit human action.

New document arrives

- > human clicks Run or invokes a command
- > system hashes files and identifies changes
 - > Docling and fallbacks parse locally
 - > local model drafts proposed spec changes
 - > Git diff and review summary generated
- > human approves or rejects
 - > approved specs become development input

The first implementation should be a deterministic Bash or Python script. An agent such as Hermes may later invoke that script as a constrained skill after a human request. A durable workflow engine can be introduced later for retries, long-running jobs, state and approval checkpoints.

Automation level	Trigger	Implementation pattern
Level 1	Human command	Deterministic local script
Level 2	Human button click	Small internal UI or approved workflow interface
Level 3	Human asks an agent	Constrained agent skill that invokes the deterministic script
Level 4	Event detected with approval gate	Durable workflow engine with audit trail, policy checks and human approval

11. GitHub Backup and Repository Strategy

As of this version, the lab is not yet backed up to GitHub. Git is installed, but the root workspace has not been initialised as a Git repository, a root .gitignore is not present, Git user details are not configured and the GitHub CLI is not installed.

The recommended immediate approach is one private root repository for the learning lab. Mature applications can later move into separate repositories.

```
enterprise-ai-lab (private GitHub repository)
docs/
configs/
shared/
projects/
README.md
opencode.json
```

Do not push:

```
OAuth credentials | auth.json | .env files | private keys
raw workplace documents | sensitive evidence extracts | model weights
```

12. Logging and Token-Usage Measurement

The experiment uses timestamped filenames so that each log can be correlated to a step and component.

```
YYYYMMDD_HHMMSS_stepXX_component_action.log
```

Examples:

```
20260607_045841_opencode-step9-debug.log
20260607_053513_step10c_workspace_structure_setup.log
20260607_053741_step10d_02-app-build-pilot_baseline_opencode_stats.log
```

Token usage must be measured per project and per milestone. Cloud and local routes should be logged separately.

Route	Metrics to capture
OpenCode with OpenAI	Project, step, phase, session count, input tokens, output tokens, cache read/write, model, reported cost and approval decisions.
Direct local Ollama	Project, step, source file, prompt tokens, output tokens, model, total duration, load duration and optional GPU utilisation snapshot.
Document ingestion	File count, pages/slides/sheets processed, extraction duration, warning count, OCR confidence and fallback usage.

13. Roadmap

Priority	Next action	Purpose
Immediate	Configure Git identity, install GitHub CLI, create a root .gitignore, initialise a root Git repository and push a private backup.	Protect the work before continuing.
Immediate	Capture a clean project-specific usage baseline.	Ensure future token deltas are meaningful.
Next	Validate the OpenAI-backed OpenCode development route with a controlled test.	Confirm the coding-agent path.
Next	Install and initialise OpenSpec.	Create a formal proposal -> specs -> design -> tasks -> implement workflow.
Next	Install Docling and fallback extractors inside the document-to-spec pilot.	Process synthetic PDFs, DOCX, PPTX, XLSX and images locally.
Next	Build a deterministic run-doc-to-spec script.	Create human-triggered local automation.
Later	Wrap the deterministic pipeline with a constrained agent skill.	Test operator-facing automation without losing control.
Later	Map all workstation shortcuts to air-gapped enterprise equivalents.	Create the enterprise deployment blueprint.

Appendix A. Current Folder Structure

```
enterprise-ai-lab/
  configs/
    opencode/
  docs/
    architecture-decisions/
```

```
checkpoints/
reports/
logs/
ingestion/
opencode/
setup/
usage/
shared/
sample-documents/
scripts/
templates/
projects/
  01-doc-to-spec-pilot/
  02-app-build-pilot/
opencode.json
README.md
```

Appendix B. Tooling Register

The tooling register is maintained locally under docs/tooling-register.md. Each new component should record its repository, licence, version, offline installation approach, telemetry behaviour, security considerations and enterprise replacement if applicable.

Decision test	Question
Licence	Can the component be legally imported, modified and operated in the target enterprise environment?
Offline operation	Can it run without internet access after controlled import?
Supply chain	Can packages, containers, model weights and vulnerability data be mirrored internally?
Observability	Can actions, model calls, tool calls, diffs and approvals be logged?
Replaceability	Can a personal-lab shortcut be swapped for an approved enterprise component without rewriting the workflow?

Appendix C. Source References

Ref	Source	URL
[1]	Docling supported formats	https://docling-project.github.io/docling/usage/supported_formats/
[2]	Docling documentation overview	https://docling-project.github.io/docling/
[3]	Docling CLI image export modes	https://docling-project.github.io/docling/reference/cli/

Ref	Source	URL
[4]	OpenCode models and local-model support	https://opencode.ai/docs/models/
[5]	OpenCode permissions	https://opencode.ai/docs/permissions/
[6]	OpenCode project rules through AGENTS.md	https://opencode.ai/docs/rules/
[7]	OpenSpec concepts: proposal -> specs -> design -> tasks -> implement	https://github.com/Fission-AI/OpenSpec/blob/main/docs/concepts.md
[8]	GitHub CLI authentication quickstart	https://docs.github.com/en/github-cli/github-cli/quickstart
[9]	GitHub CLI repository creation	https://cli.github.com/manual/gh_repo_create
[10]	GitHub remote repositories	https://docs.github.com/articles/about-remote-repositories

Note: This report is an evergreen snapshot. Update the version history, completed learning points, architecture decisions and roadmap after each validated milestone.